



Configuration d'hébergement — application complète « ADM - Gestion Planning »

Ce document décrit ce qu'il faut prévoir pour héberger **l'ensemble** de l'application : API Node.js (Express), base **PostgreSQL**, fichiers uploadés, tâches planifiées (cron), temps réel (Socket.IO), e-mails, notifications push optionnelles, et le **frontend** (build statique ou application mobile Capacitor).

1. Architecture cible (recommandée)

Composant	Rôle
PostgreSQL	Données applicatives (Prisma)
Backend Node.js	API REST <code>/api/*</code> , fichiers <code>/uploads</code> , WebSocket <code>/socket.io</code> , Swagger <code>/api/docs</code> , santé <code>/health</code>
Reverse proxy (Nginx, Caddy, Traefik, équivalent PaaS)	HTTPS, en-têtes de sécurité complémentaires, upgrade WebSocket vers le backend

Composant	Rôle
Frontend	Fichiers statiques après <code>npm run build</code> (Vite), ou hébergement derrière le même domaine sous un préfixe

Deux schémas courants :

1. **Sous-domaines** — `https://app.exemple.fr` (frontend) et `https://api.exemple.fr` (backend). Le frontend utilise `VITE_API_URL=https://api.exemple.fr` au moment du build.
2. **Même origine** — le proxy sert le frontend sur `/` et proxifie `/api`, `/uploads`, `/socket.io` vers le backend. Dans ce cas, `VITE_API_URL` peut rester vide si l'URL publique du site est la même que celle du navigateur (le client utilisera l'origine courante pour l'API et le socket).

2. Prérequis techniques

Élément	Détail
Node.js	Version 20 LTS ou 22 LTS recommandée (pas de champ engines imposé dans le dépôt ; vérifier la compatibilité avec Prisma 5 et Vite 8).

Élément	Détail
PostgreSQL	Version récente compatible Prisma (14+ en général).
Outils système (sauvegardes)	Binaires <code>pg_dump</code> et <code>psql</code> accessibles sur le serveur si vous utilisez les sauvegardes planifiées intégrées (voir variables <code>PG_DUMP_PATH</code> / <code>PSQL_PATH</code>).
Disque persistant	Répertoires <code>backend/uploads/</code> et <code>backend/backups/</code> (créés au démarrage si possible d'écrire sur le disque). Sans volume persistant, avatars, pièces jointes et sauvegardes sont perdus au redéploiement.
Réseau	Le proxy doit autoriser WebSocket (Socket.IO) vers le même hôte/port que l'API.

2.1 Configuration minimum du serveur d'hébergement

Cibles indicatives pour une **petite équipe** (quelques dizaines d'utilisateurs actifs, trafic modéré). Monter en charge si la base est volumineuse, si les pièces jointes sont nombreuses ou si plusieurs instances du backend tournent en parallèle.

Machine applicative (backend + reverse proxy sur le même VPS)

Ressource	Minimum recommandé	Commentaire
vCPU	1 cœur	Suffisant pour démarrer ; prévoir 2 si Socket.IO, exports et sauvegardes coïncident souvent.
RAM	2 Go	Node + Express + Prisma ; 4 Go plus confortable dès que la charge ou les jobs cron sont visibles.
Stockage	20 Go SSD	Système, <code>node_modules</code> , journaux, <code>`uploads/`</code> et <code>`backups/`</code> ; augmenter selon fichiers joints et rétention des dumps.
Système	Linux x86_64 (ex. Ubuntu 22.04 ou 24.04 LTS)	Stack documentée pour ce type d'hébergement ; macOS/Windows possibles en dev, moins typiques en prod.
Réseau sortant	SMTP (ports 587/465), optionnel HTTPS vers APIs externes	À autoriser par pare-feu si les politiques sont strictes.

Base PostgreSQL

Scénario	Minimum recommandé
PostgreSQL managé (RDS, Scaleway, Supabase, etc.)	Plan le plus petit du fournisseur (souvent ~1 Go RAM pour des bases légères) ; dimensionner le stockage selon la croissance.
PostgreSQL sur la même machine que l'API	Ajouter ~512 Mo à 1 Go RAM réservée à Postgres au minimum ; 4 Go RAM totales sur le VPS est un plancher réaliste pour API + Postgres + OS.

Pare-feu et ports

Port	Direction	Usage
443 (TCP)	Entrant public	HTTPS (et WebSocket sur la même URL).
80 (TCP)	Entrant public	Redirection HTTP → HTTPS (optionnel mais courant).
3001 (ou PORT)	Interne seulement	Backend Node ; le proxy y envoie le trafic, ne pas exposer sur Internet si possible.

Port	Direction	Usage
5432	Interne seulement	PostgreSQL ; accessible uniquement depuis le backend (ou réseau privé / VPC).

Système et fiabilité

- **Horloge synchronisée (NTP)** : importante pour les crons, les logs et la 2FA (TOTP).
- **Espace disque surveillé** : les sauvegardes PostgreSQL et les uploads grossissent ; prévoir alertes avant saturation.
- **Fichier swap** (1–2 Go) : utile sur petit VPS pour éviter les OOM lors des pics (sacrifie un peu de latence).

Frontend statique

Les fichiers `dist/` peuvent être servis par le **même** serveur (Nginx sur la même VM) ou par un CDN ; le minimum matériel est alors absorbé par la VM du proxy. Un hébergement **objet** + **CDN** (S3, etc.) n'impose pas de VPS dédié au frontend.

3. Déploiement du backend

À exécuter dans le dossier `backend/` :

```
npm ci
```

```
npx prisma generate
npx prisma migrate deploy # ou db push selon votre politique de
migrations
npm run start # production : node server.js
```

- Fichier d'entrée : `server.js` (script `start` dans `package.json`).
- Port d'écoute : variable ``PORT`` (défaut **3001**).
- Santé : requête **GET** ``/health`` pour les sondes de charge équilibreur / orchestrateur.

4. Déploiement du frontend (web)

Dans `frontend/` :

```
npm ci
npm run build
```

- Sortie : dossier ``frontend/dist/`` (fichiers statiques à servir par Nginx, S3+CloudFront, Netlify, Vercel, etc.).
- **Obligatoire en production web / mobile** : définir ``VITE_API_URL`` au build avec l'URL **publique HTTPS** du backend **sans** suffixe `/api` (ex. `https://api.exemple.fr`). Les appels iront vers `.../api` automatiquement.
- Référence : `frontend/.env.example`.

5. Reverse proxy : points indispensables

3. **TLS (HTTPS)** partout en production (`NODE_ENV=production` active notamment CSP avec `upgrade-insecure-requests`).
4. **CORS** : le backend utilise ``FRONTEND_URL`` comme origine autorisée. Si plusieurs origines (web + outils), séparez-les par des **virgules** ; la même chaîne sert aussi aux origines Socket.IO côté serveur.
5. **WebSocket** : pour `/socket.io`, conserver les en-têtes `Upgrade` et `Connection` (exemple Nginx : `proxy_http_version 1.1, proxy_set_header Upgrade $http_upgrade, proxy_set_header Connection "upgrade"`).
6. **Taille des corps** : prévoir un `client_max_body_size` (ou équivalent) suffisant pour les uploads (multer côté API).
7. **Timeouts** : opérations longues (exports, sauvegardes) peuvent nécessiter des timeouts proxy plus élevés que la valeur par défaut.

6. Variables d'environnement — backend

Créer un fichier `.env` sur le serveur (ne jamais commiter les secrets). Les variables ci-dessous sont celles réellement lues dans le code du dépôt.

6.1 Obligatoires en production

Variable	Description
<code>NODE_ENV</code>	Mettre <code>`production`</code> .

Variable	Description
DATABASE_URL	URL PostgreSQL Prisma, ex. postgresql://USER:PASSWORD@HOST:5432/DB?schema=public
JWT_SECRET	Secret fort et stable pour signer les JWT (accès, refresh, liens magiques, socket). Ne pas changer en prod sans invalider les sessions.
FRONTEND_URL	URL(s) du frontend pour CORS, CSP connectSrc, liens dans les e-mails, reset mot de passe, etc. Plusieurs origines : liste séparée par des virgules (sans espaces superflus), ex. https://app.exemple.fr,https://www.exemple.fr
SWAGGER_USER	Identifiant Basic Auth pour /api/docs (requis si NODE_ENV=production).
SWAGGER_PASSWORD	Mot de passe Basic Auth pour /api/docs.

6.2 Fortement recommandées

Variable	Défaut / note
PORT	3001

Variable	Défaut / note
BACKEND_URL	URL publique absolue du backend (ex. liens dans certaines routes réunions). Sinon dérivé de <code>http://localhost:\${PORT}</code> .
JWT_EXPIRY	Durée access token, ex. 15m
JWT_REFRESH_EXPIRY	Durée refresh token, ex. 7d
JWT_ACTIVATION_EXPIRY	Jetons d'activation compte, ex. 7d

6.3 E-mail (SMTP)

Sans configuration correcte, l'envoi peut viser `localhost:1025` (inadapté en production).

Variable	Description
SMTP_HOST	Hôte du relais SMTP
SMTP_PORT	Port (souvent 587 ou 465)
SMTP_SECURE	true si TLS direct (souvent port 465)

Variable	Description
SMTP_USER / SMTP_PASS	Authentification si requise
SMTP_FROM	Adresse expéditeur (ex. Planning <noreply@exemple.fr>)

6.4 Sécurité et limites

Variable	Défaut
MAX_LOGIN_ATTEMPTS	5
LOGIN_BLOCK_DURATION	15 (minutes)
MAX_2FA_ATTEMPTS	5

6.5 Sauvegardes PostgreSQL (cron intégré)

Variable	Description
DISABLE_BACKUP_CRON	true pour désactiver le cron de backup
BACKUP_CRON_EXPRESSION	Expression cron (défaut 0 0 * * 0 — chaque dimanche minuit)
BACKUP_CRON_TIMEZONE	Fuseau IANA optionnel (ex. Europe/Paris)
PG_DUMP_PATH / PSQL_PATH	Chemins binaires si non dans le PATH
BACKUP_TIMEOUT_MS / BACKUP_RESTORE_TIMEOUT_MS	Timeouts des commandes
BACKUP_NOTIFY_EMAIL	E-mail de notification en cas d'événements de backup

Les fichiers sont écrits sous ``backend/backups/`` (à monter en volume persistant).

6.6 Autres

Variable	Usage
APP_NAME	Libellé dans le rapport hebdomadaire

Variable	Usage
WEEKLY_REPORT_EMAIL	Destinataire du rapport hebdo (défaut codé en dur dans le job si non défini — à surcharger en prod)
PUBLIC_APP_URL	Fallback pour certaines URLs « publiques » dans les e-mails si FRONTEND_URL absent
EMAIL_LOGO_URL	URL absolue d'un logo pour les e-mails

7. Variables d'environnement — frontend (build)

Variable	Quand
VITE_API_URL	Requis dès que le frontend n'est pas servi avec le proxy Vite vers localhost:3001 (production web, preview, build Capacitor). Ex. https://api.exemple.fr

Les variables Vite doivent être présentes **au moment de `npm run build`** (injectées en dur dans le bundle).

8. Tâches planifiées (cron dans le processus Node)

Le serveur enregistre des crons au démarrage : rapport hebdomadaire, rappels réunions, digest notifications, auto-fermeture réunions, sauvegarde DB (si activée).

Une seule instance du backend doit exécuter ces crons en production, ou bien désactiver les crons sur les réplicas et externaliser les tâches (scheduler externe) si vous scalez horizontalement — le code actuel ne distingue pas les rôles leader/worker.

9. Temps réel (Socket.IO)

- Le client se connecte à la même base que `VITE_API_URL`` (ou à l'origine du site si vide en mode relatif).
- Le serveur accepte les origines dérivées de `FRONTEND_URL`` (liste séparée par virgules).
- Le proxy doit laisser passer **WebSocket** vers le backend.

10. Application mobile (Capacitor)

- Après modification de `VITE_API_URL`, exécuter `npm run build`` puis `npx cap sync`` (scripts `cap:sync` dans `frontend/package.json`).
- Android : `google-services.json` et configuration FCM côté Firebase pour les push.
- `capacitor.config.json` définit notamment `server.hostname` et `androidScheme` ; l'API doit être en **HTTPS** (pas de contenu mixte si `allowMixedContent` reste à `false`).

11. Checklist avant mise en ligne

- [] PostgreSQL provisionné, `DATABASE_URL` défini, migrations appliquées (`prisma migrate deploy`).
- [] `JWT_SECRET` généré (long, aléatoire) et stocké dans un gestionnaire de secrets.
- [] `FRONTEND_URL` aligné sur l'URL réelle du navigateur (protocole + hôte + port si non standard).
- [] `VITE_API_URL` défini au build du frontend si API sur un autre hôte.
- [] HTTPS sur le proxy ; WebSocket testé (messagerie / présence).
- [] SMTP de production testé (invitation, reset mot de passe, rapports).
- [] `SWAGGER_USER` / `SWAGGER_PASS` définis si vous gardez `/api/docs` accessible.
- [] Volume persistant pour `uploads/` et `backups/`.
- [] `pg_dump` disponible si backups automatiques activées.
- [] Sonde ``GET /health`` configurée sur l'orchestrateur.

12. Exemple minimal `.env` backend (à adapter)

```
NODE_ENV=production
PORT=3001
```

```
DATABASE_URL=postgresql://user:pass@db.internal:5432/gestion_planning?s
chema=public
JWT_SECRET=REPLACER_PAR_UN_SECRET_LONG_ET_ALEATOIRE
FRONTEND_URL=https://app.exemple.fr
BACKEND_URL=https://api.exemple.fr
SMTP_HOST=smtp.exemple.fr
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=noreply@exemple.fr
SMTP_PASS=***
SMTP_FROM=Gestion Planning <noreply@exemple.fr>
SWAGGER_USER=admin_docs
SWAGGER_PASS=***
JWT_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
```

Exemple build frontend :

```
VITE_API_URL=https://api.exemple.fr npm run build
```

Document généré à partir de la structure du dépôt (Express, Prisma PostgreSQL, Vite, Socket.IO, Nodemailer, Firebase Admin optionnel). Ajuster les noms de domaine, secrets et politiques de sauvegarde selon votre hébergeur.